

1. Introdução

Este relatório apresenta o desenvolvimento da API REST **DeliveryTech**, criada durante os estudos de Java e Spring Boot.

O objetivo do projeto foi desenvolver uma aplicação backend capaz de simular o funcionamento de plataformas de delivery como iFood, Uber Eats e Rappi, aplicando conceitos utilizados no mercado de desenvolvimento de software.

Durante o desenvolvimento foram utilizados diversos recursos do ecossistema Spring, como autenticação JWT, Spring Security, persistência de dados com JPA, documentação automática com Swagger, monitoramento da aplicação e testes automatizados.

Além das tecnologias estudadas durante o curso, parte da evolução do projeto contou com o auxílio de ferramentas de Inteligência Artificial para pesquisas, revisão de código, sugestões de arquitetura e melhoria contínua da documentação, sempre com validação e adaptação manual durante o desenvolvimento.

2. Objetivos do Projeto

O principal objetivo foi construir uma API REST organizada, segura e escalável.

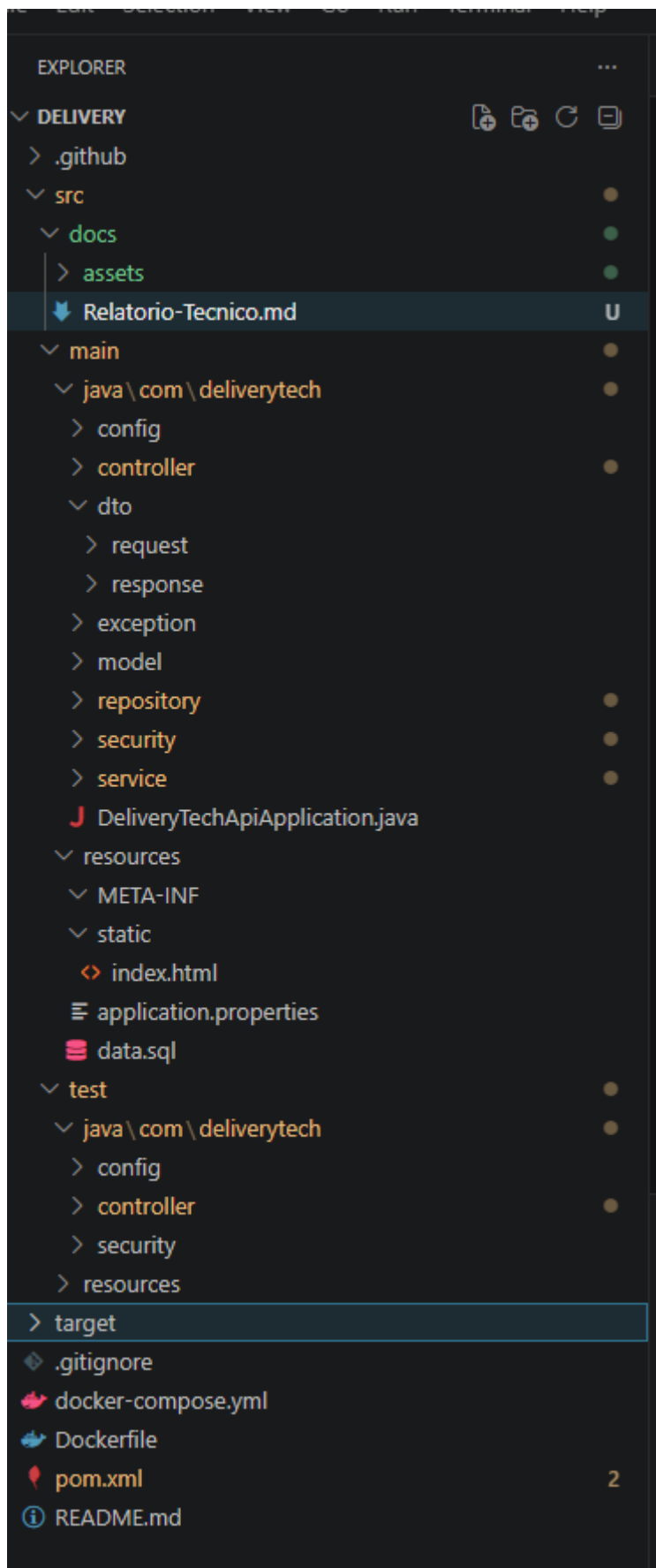
Entre os objetivos específicos destacam-se:

- aplicar conceitos de arquitetura em camadas;
- implementar autenticação utilizando JWT;
- controlar permissões através de perfis de usuários;
- desenvolver operações completas de CRUD;
- documentar a API utilizando Swagger;
- realizar testes automatizados;
- preparar a aplicação para utilização com Docker.

3. Arquitetura da Aplicação

O projeto foi desenvolvido utilizando arquitetura em camadas.

Cada camada possui uma responsabilidade específica.



As principais camadas são:

Controller

Responsável por receber as requisições HTTP e devolver as respostas da API.

Service

Contém toda a regra de negócio da aplicação.

Repository

Responsável pelo acesso ao banco utilizando Spring Data JPA.

Database

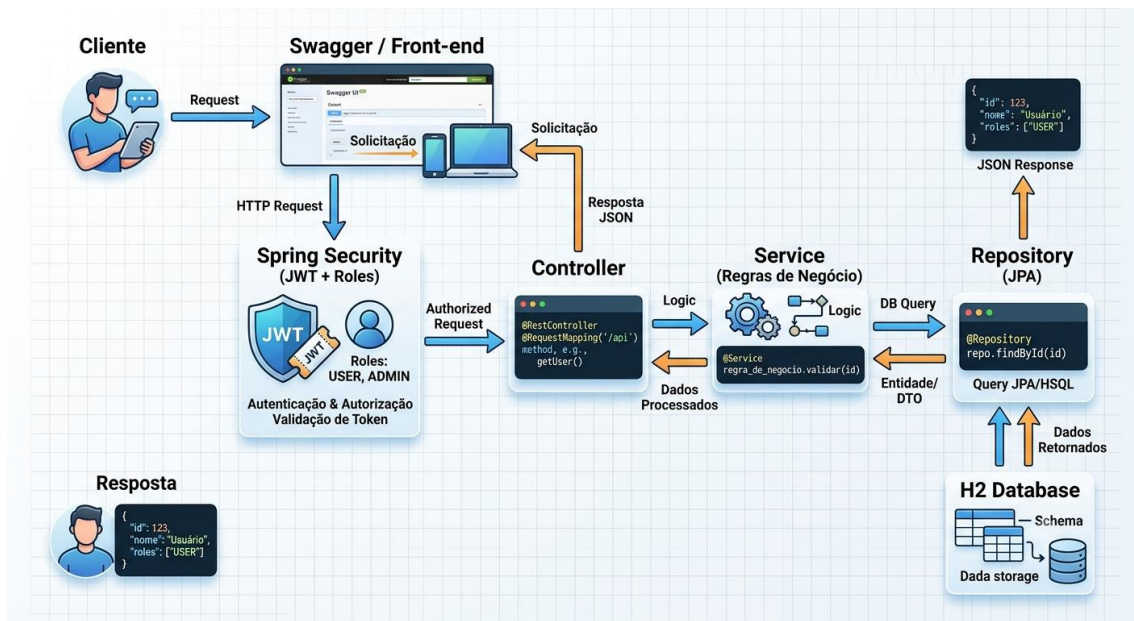
Responsável pelo armazenamento dos dados.

Essa divisão facilita manutenção, testes e futuras evoluções.

4. Funcionamento da API

O fluxo de uma requisição ocorre da seguinte forma:

1. Cliente realiza uma requisição HTTP.
2. O Spring Security verifica autenticação.
3. Caso o token seja válido, a requisição segue.
4. O Controller recebe a chamada.
5. O Service executa as regras de negócio.
6. O Repository consulta ou grava informações.
7. A resposta retorna ao cliente.



5. Banco de Dados

Durante o desenvolvimento foi utilizado o banco H2 Database.

Para facilitar os testes, foi criado um arquivo chamado **data.sql**.

Sempre que a aplicação é iniciada, esse arquivo popula automaticamente o banco com registros de clientes, restaurantes e produtos.

Isso permite que qualquer pessoa execute a API e já tenha dados disponíveis para realizar testes imediatamente.

6. Segurança da Aplicação

A autenticação foi implementada utilizando **JWT (JSON Web Token)**.

O funcionamento ocorre da seguinte forma:

- usuário realiza login;
- a API valida as credenciais;
- um Token JWT é gerado;
- o token deve ser enviado nas próximas requisições utilizando o Header Authorization.

Authorization: Bearer TOKEN

Atualmente esse processo é utilizado para usuários administradores e clientes.

Como melhoria futura, pretende-se implementar um sistema semelhante às plataformas atuais, permitindo login através de e-mail e senha com geração automática do token após autenticação.

7. Perfis de Usuário (Roles)

O sistema utiliza controle de acesso baseado em perfis.

ADMIN

Possui acesso completo ao sistema.

Pode cadastrar, editar, excluir e consultar todas as informações.

CLIENTE

Pode realizar consultas, efetuar pedidos e acessar funcionalidades destinadas ao usuário final.

RESTAURANTE

Perfil reservado para gerenciamento dos restaurantes cadastrados.

ENTREGADOR

Perfil previsto para futuras versões da aplicação.

Será responsável pelo gerenciamento das entregas.

8. Documentação da API

Toda a documentação foi gerada automaticamente utilizando Swagger.

Através da interface é possível:

- visualizar endpoints;
- testar requisições;
- enviar parâmetros;
- visualizar respostas.

9. Monitoramento

Foi utilizado o Spring Boot Actuator.

Essa ferramenta disponibiliza informações importantes sobre a aplicação, como:

- saúde da aplicação;
- métricas;
- informações do ambiente.

10. Testes

Durante o desenvolvimento foram implementados diferentes tipos de testes.

Entre eles:

- Testes Unitários;
- Testes de Integração;
- Testes de Segurança.

Todos podem ser executados utilizando:

`mvn test`

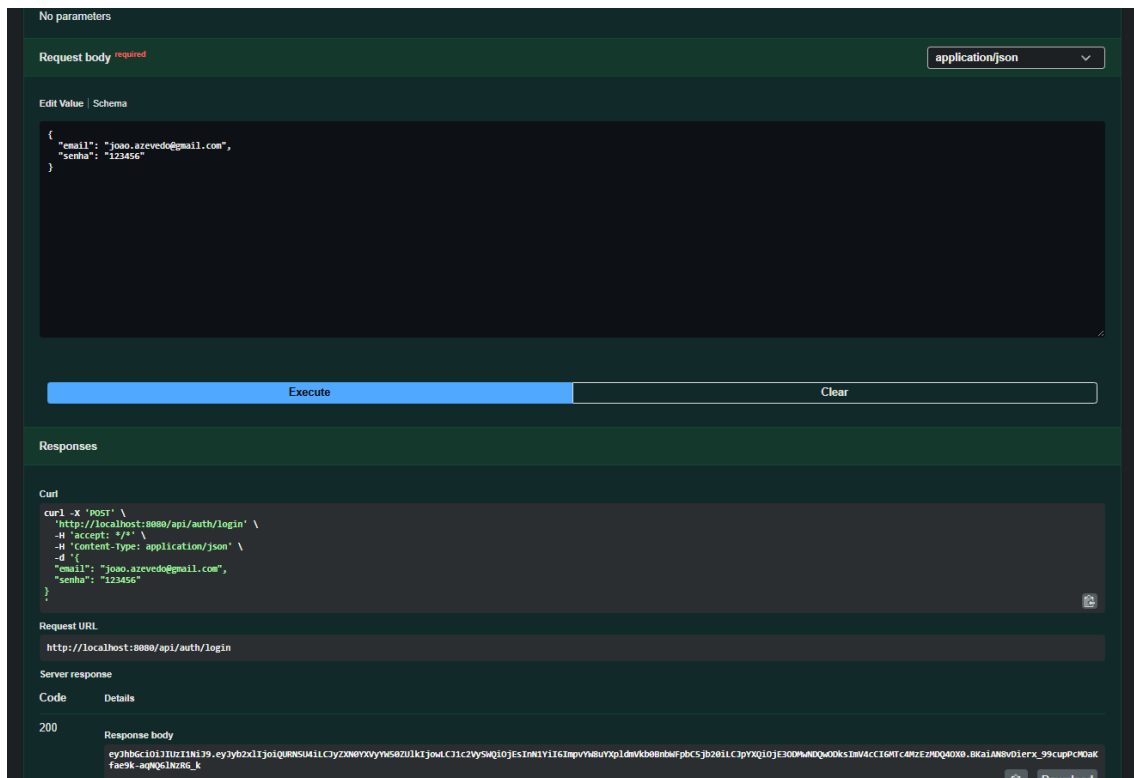
```
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.491 s -- in com.deliverytech.security.SecurityIntegrationTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 30, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 38.585 s
[INFO] Finished at: 2026-07-03T19:23:43-03:00
[INFO] -----
C:\Users\Particular\delivery>
```

Atualmente temos 30 testes, rodando no sistema, com possibilidade de serem implementando mais no futuro.

11. Exemplos de Funcionamento

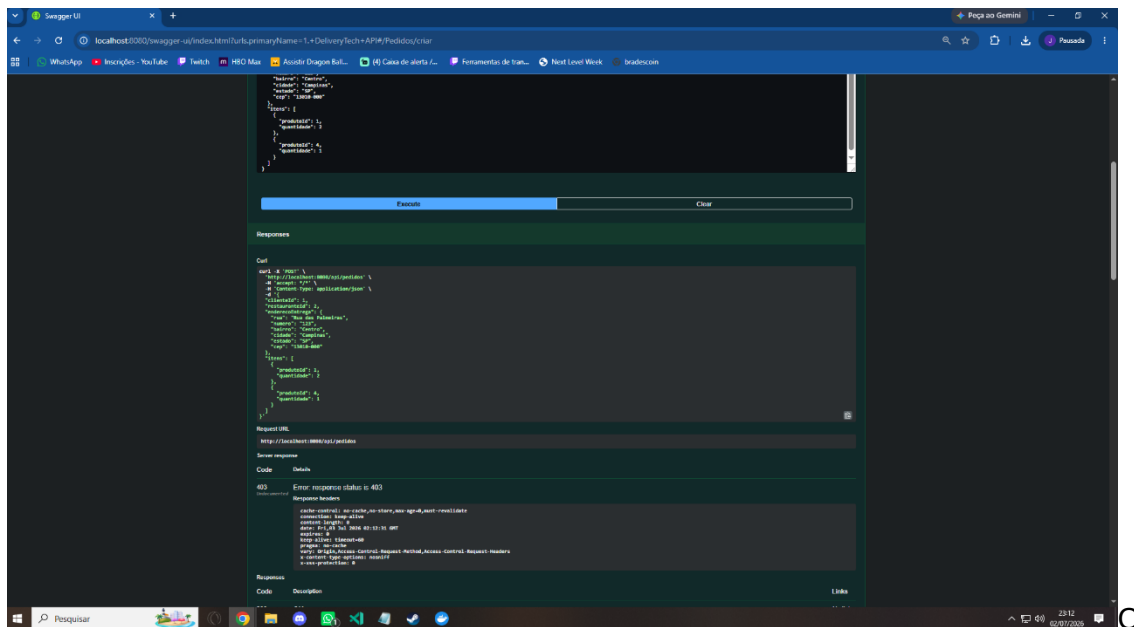
Nesta seção são apresentados alguns exemplos de utilização da API.

Cadastro realizado com sucesso



Entrada autorizada como ADMIN.

Erro 403



código HTTP 403 indica que o usuário foi autenticado, porém não possui permissão para acessar aquele recurso.

Erro 404

- PostgreSQL;
- Redis;
- RabbitMQ;
- Flyway;
- Kubernetes;
- Deploy em nuvem;
- Dashboard administrativo;
- Sistema completo de entregadores;
- Cadastro de motos e veículos;
- Rastreamento em tempo real;
- Sistema de avaliações;
- Pagamentos online;
- Notificações em tempo real;
- Histórico completo dos pedidos.

Essas melhorias aproximariam a aplicação do funcionamento de plataformas comerciais de delivery.

13. Conclusão

O desenvolvimento da API DeliveryTech permitiu aplicar, na prática, conceitos importantes do desenvolvimento backend utilizando Java e Spring Boot.

Durante o projeto foram estudados temas como arquitetura em camadas, segurança, autenticação JWT, documentação de APIs, persistência de dados, testes automatizados e containerização com Docker.